



土制 Nix S3 Binary Cache

Homemade Nix S3 Binary Cache

Yinfeng

2026-06-14



本作品采用署名-非商业性使用-相同方式共享 4.0 协议国际版协议授权。

关于我

Yinfeng



github.com/linyinfeng

Nix 相关项目：

- [nix-cache-overlay](#) - S3 cache 代理
- [nix-gc-s3](#) - 为 S3 cache 做 GC
- [dotfiles](#) - 我的配置文件
- [commit-notifier](#) - NixOS CN 群的 PR 通知
- [angrr](#) - 自动 Nix GC Roots 管理
- [flat-flake](#) - 扁平化 flake inputs 检查
- [oranc](#) - OCI registry as Nix cache (实验性)

简介

一个我已经使用了三年多的 Nix S3 Binary Cache 方案。

1. **土制灵车**
2. **极低成本**：使用 Cloudflare R2 等有免费额度的 S3 服务，每月 **0 USD**
3. **几乎 Serverless**：稳定，高性能，无服务器瓶颈
4. **极佳兼容性**：使用 `nix copy` 直接上传

注：第一点与其他特色并没有冲突。

目标

1. 增进大家对 Nix binary cache 的了解
2. 向大家介绍我写的两个简单的小工具：
 - [github:linyinfeng/nix-cache-overlay](#)
 - [github:linyinfeng/nix-gc-s3](#)
3. 抛砖引玉

Nix binary cache 简介

Nix 客户端如何利用 HTTP binary cache?

1. 当 Nix 要构建某个 derivation 的输出时，通常¹ Nix 计算出输出的 store path，例如：

```
1 $ nix eval nixpkgs#hello^out --raw
```

console

```
2 /nix/store/zi2bj2hlavv8q743li2s9diqbcpmrf9b-hello-2.12.3
```

¹非 content-addressed derivation。

Nix binary cache 简介

2. Store path

`/nix/store/zi2bj2hlavv8q743li2s9diqbcpmrf9b-hello-2.12.3`

中的 hash `zi2bj2hlavv8q743li2s9diqbcpmrf9b` 被认为是不会重复的，这是 binary cache 的索引。

Nix 随即尝试从 substituters 下载以下文件：

`zi2bj2hlavv8q743li2s9diqbcpmrf9b.narinfo`

Nix binary cache 简介

curl <https://cache.nixos.org/zi2bj2hlavv8q743li2s9diqbcpmrf9b.narinfo>

```
1  StorePath: /nix/store/zi2bj2hlavv8q743li2s9diqbcpmrf9b-hello-2.12.3
2  URL: nar/1zzwzcsbpsghsbfdw416dgmfankjs4chksx1cic1p1z8v6vr0s8.nar.xz
3  Compression: xz
4  FileHash: sha256:1zzwzcsbpsghsbfdw416dgmfankjs4chksx1cic1p1z8v6vr0s8
5  FileSize: 58480
6  NarHash: sha256:0vwm6sr61cx6hydqlx3phhg1a0830k61dbfwqlkhilkp cbppjmdw
7  NarSize: 279624
8  References: 57iz36553175g3178pvxjij8z5rcsd4n-glibc-2.42-61 zi2bj2hlavv8q743li2s9diqbcpmrf9b-
hello-2.12.3
9  Deriver: 67mdzby3g0maqpp93xj03rc99nnrpd9-hello-2.12.3.drv
10 Sig: cache.nixos.org-1:DwOHEUMyxq4aUrwzcZJrPPqqlImdA7042VJ+HW0nyjZeBMaKkSQxFS2vArnR...
```

Nix binary cache 简介

3. 根据 Sig 验证签名后，Nix 下载

URL: `nar/1zzwzcsbpsghsbfdw416dgmfankjs4chksx1cic1p1z8v6vr0s8.nar.xz`

解压并加入 store。

可以注意到 nar 的路径中也有一个 hash，这个 hash 是 nar 文件本身的 hash，也就是说，它“content-addressed”，与 store path 里的 hash 并不相同。

4. cache.nixos.org CDN 的上游其实²直接是一个 AWS S3 服务的 public URL。

²NixOS 的基础设施配置是很透明公开的，见：[NixOS/infra - terraform/cache.tf](https://github.com/NixOS/infra-terraform/blob/master/cache.tf)。

最简单的 Nix S3 Binary Cache

`nix copy` 命令支持拷贝到各种“Nix store”。而 S3 binary cache 也是一种 Nix store，URL 形如 `s3://BUCKET_NAME`。

支持任何 AWS S3 兼容服务：Cloudflare R2、Backblaze B2、自建 Garage 等等。

```
1 export AWS_ACCESS_KEY_ID="..."
2 export AWS_SECRET_ACCESS_KEY="..."
3 # AWS
4 nix copy "nixpkgs#hello" --to "s3://$BUCKET_NAME"
5 # 非 AWS
6 export AWS_EC2_METADATA_DISABLED=true
7 nix copy "nixpkgs#hello" --to "s3://$BUCKET_NAME?endpoint=$ENDPOINT_URL"
```

bash

签名

通常我们会想要给 cache 签名，可以直接使用 `nix store sign` 在本地签名，然后 `nix copy`。

```
1 # 生成密钥
2 nix key generate-secret --key-name "$KEY_NAME"
3 # 签名
4 nix store sign "nixpkgs#hello" --recursive --key-file "$SECRET_KEY_FILE"
5 # 上传
6 nix copy "nixpkgs#hello" --to "s3://$BUCKET_NAME?endpoint=$ENDPOINT_URL"
```

bash

导出公钥：

```
1 cat "$SECRET_KEY_FILE" | nix key convert-secret-to-public
```

bash

公钥形如：cache.li7g.com:YIVuYf8Ajn0c5oncjClmtM19RaAZf0KLFFyZUp0rfqM=

我的配置

使用 Cloudflare R2 + 自定义域名 cache.li7g.com。

```
1  resource "cloudflare_r2_bucket" "cache" {
2      account_id      = ...
3      name             = "cache-li7g-com"
4      location        = "APAC"
5      storage_class    = "Standard"
6  }
7  resource "cloudflare_r2_custom_domain" "cache" {
8      account_id      = ...
9      enabled         = true
10     bucket_name     = cloudflare_r2_bucket.cache.name
11     domain           = "cache.li7g.com"
12     zone_id          = cloudflare_zone.com_li7g.id
13 }
```

terraform

使用搭建好的 cache

使用 cache 只需要通过任意方式指定好 Nix 选项：

```
1 substituters = ... https://cache.li7g.com
```

ini

```
2 trusted-public-keys = ...  
  cache.li7g.com:YIVuYf8Ajn0c5oncjClmtM19RaAZf0KLFFyZUp0rfqM=
```

忽略上游已有内容

nix copy 上传时**不会检查上游**，比如 cache.nixos.org 中是否已有相同内容。

我的主力机 closure 大小：

```
1 $ nix path-info /run/current-system \  
2     --closure-size --human-readable  
3 /nix/store/...-nixos-system-parrot-26.05... 35.5 GiB
```

console

35.5 GiB，远超 R2 免费额度（10 GB）。

忽略上游已有内容

nix copy 上传时**不会检查上游**，比如 cache.nixos.org 中是否已有相同内容。

我的主力机 closure 大小：

```
1 $ nix path-info /run/current-system \  
2     --closure-size --human-readable  
3 /nix/store/...-nixos-system-parrot-26.05... 35.5 GiB
```

console

35.5 GiB，远超 R2 免费额度（10 GB）。

需要避免重复存储上游已有内容。

nix copy 的行为

nix copy 在上传前会检查目标 cache 中是否已有 narinfo 文件：

1. 如果 store path 文件名为： /nix/store/*i3zw7h6p...4v5w*-hello-2.12.2
2. narinfo 文件名： *i3zw7h6p...4v5w*.narinfo
3. 通过 S3 GetObject 检查该文件是否存在（就是个 HTTP GET）

nix copy 的行为

nix copy 在上传前会检查目标 cache 中是否已有 narinfo 文件：

1. 如果 store path 文件名为： /nix/store/*i3zw7h6p...4v5w*-hello-2.12.2
2. narinfo 文件名： *i3zw7h6p...4v5w*.narinfo
3. 通过 S3 GetObject 检查该文件是否存在（就是个 HTTP GET）

从上游 cache 偷个 narinfo 文件，返回给 Nix 客户端，就能避免重复上传。

nix-cache-overlay – 轻量 nix copy 代理服务器

[github:linyinfeng/nix-cache-overlay](https://github.com/linyinfeng/nix-cache-overlay)

做两件事：

1. 把 narinfo 的 GET/HEAD 请求转发到上游 cache
 - 上游返回 200 → 返回给客户端
 - 上游返回 404 → 尝试下一个上游
2. 所有上游都 404 → 认证后签上 SigV4 签名，转发到 S3 服务器

nix-cache-overlay – 轻量 nix copy 代理服务器

[github:linyinfeng/nix-cache-overlay](https://github.com/linyinfeng/nix-cache-overlay)

认证方式：忽略签名，把 AWS_ACCESS_KEY_ID 当 token 使用。

代价是降低了一些安全性，好处是不用受 SigV4 的折磨（大雾

nix-cache-overlay – 轻量 nix copy 代理服务器

[github:linyinfeng/nix-cache-overlay](https://github.com/linyinfeng/nix-cache-overlay)

做两件事：

1. 把 narinfo 的 GET/HEAD 请求转发到上游 cache
 - 上游返回 200 → 返回给客户端
 - 上游返回 404 → 尝试下一个上游
2. 所有上游都 404 → 认证后签上 SigV4 签名，转发到 S3 服务器

因为它只是一个简单的代理服务器，所以它兼容性拉满，直接支持 S3 multipart 上传，还可以直接支持上传构建日志等特殊功能。

nix-cache-overlay 配置

引入项目的 Nixpkgs overlay 与 NixOS 模块后，NixOS module 配置：

```
1 {  
2   services.nix-cache-overlay = {  
3     enable = true;  
4     listen = "[::1]:8080";  
5     endpoint = "https://host.of.s3.endpoint";  
6     environmentFile = /path/to/env/file;  
7   };  
8 }
```

nix

nix-cache-overlay 配置

environmentFile 内容:

```
1 # 用于代理连接 S3 的认证
```

txt

```
2 AWS_ACCESS_KEY_ID=...
```

```
3 AWS_SECRET_ACCESS_KEY=...
```

```
4
```

```
5 NIX_CACHE_OVERLAY_TOKEN=... # 用于客户端连接代理的认证
```

```
6
```

```
7 AWS_EC2_METADATA_DISABLED=true # 非 AWS 需要
```

使用 nix-cache-overlay

```
1 export AWS_ACCESS_KEY_ID="$NIX_CACHE_OVERLAY_TOKEN"
2 export AWS_SECRET_ACCESS_KEY="-" # 不重要
3 export AWS_EC2_METADATA_DISABLED=true
4 nix store sign "nixpkgs#hello" --recursive --key-file "$SECRET_KEY_FILE"
5 nix copy "nixpkgs#hello" --to "s3://$BUCKET_NAME?endpoint=http://[::1]:8080"
```

bash

注意： nix copy 对 narinfo 查询的并发非常恐怖，建议把 overlay 部署在本地，不要通过某些多进程架构的反向代理访问，会把反代打爆。

```
1 Jan 22 23:30:21 nuc nginx[85266]: [alert] 512 worker_connections are not enough
```

log

我的配置

使用 nix-cache-overlay 后，存储我一堆机器的系统的 binary cache 只需要不到 6 GiB 的存储空间。

```
1 $ mc du r2-cache/cache-li7g-com
2 5.7GiB 14352 objects cache-li7g-com
```

console

目前每个月 Cloudflare 账单都是 0 USD。

GC

Nix 本身只支持上传 S3 binary cache，但不支持 GC，需要自己实现。

GC 其实分两部分：

1. 管理 GC roots
2. 根据 GC roots 进行 GC

管理 GC roots

我的方案：正好我有自建的 Hydra CI³，直接让它来管理 GC roots。

- Hydra 提供了 GC roots 目录
- Job set 的 “Number of evaluations to keep” 配置⁴控制保留数量
- Binary cache 只用来存储 hydra 构建的内容，不做随意的 push

³即 NixOS 的官方 CI [github:NixOS/hydra](https://github.com/NixOS/hydra)。

⁴Hydra 声明式配置中的 `keepnr`。

根据 GC roots 进行 GC

理论上，应该做一个 tracing GC，解析 narinfo 文件获取依赖列表，递归找到所有可达的 store path。典型的 narinfo 文件：

```
1 StorePath: /nix/store/i3zw7h6p...-hello-2.12.2 plain  
2 URL: nar/0jra6lgd...nar.xz  
3 References: i3zw7h6p...-hello-2.12.2 j193mfi0...-glibc-2.40-66  
4 Sig: cache.nixos.org-1:K25JMfP0...
```

Hydra 服务器上，closure 本地都有，直接用 `nix-store --query --requisites` 判断依赖关系，比较快。最后调用 S3 API 批量删除（一次最多 1000 个对象）。

代码见 [linyinfeng/nix-gc-s3](https://github.com/linyinfeng/nix-gc-s3)。

一个经典的坑

我们提到过， nar 文件的路径中的 hash 是通过 nar 文件本身的内容计算出来的。

```
1 StorePath: /nix/store/zi2bj2hlavv8q743li2s9diqbcpmrf9b-hello-2.12.3 txt  
2 URL: nar/1zzwzcsbpsghsbfjdw416dgmfankjs4chksx1cic1p1z8v6vr0s8.nar.xz  
3 ...
```

一个经典的坑

我们提到过， nar 文件的路径中的 hash 是通过 nar 文件本身的内容计算出来的。

```
1 StorePath: /nix/store/zi2bj2hlavv8q743li2s9diqbcpmrf9b-hello-2.12.3 txt  
2 URL: nar/1zzwzcsbpsghsbfjdw416dgmfankjs4chksx1cic1p1z8v6vr0s8.nar.xz  
3 ...
```

因此完全可能出现多个 narinfo 文件的 URL 指向同一个 nar 文件。

一个经典的坑

我们提到过， nar 文件的路径中的 hash 是通过 nar 文件本身的内容计算出来的。

```
1 StorePath: /nix/store/zi2bj2hlavv8q743li2s9diqbcpmrf9b-hello-2.12.3 txt  
2 URL: nar/1zzwzcsbpsghsbfjdw416dgmfankjs4chksx1cic1p1z8v6vr0s8.nar.xz  
3 ...
```

因此完全可能出现多个 narinfo 文件的 URL 指向同一个 nar 文件。

所以 GC 时，必须下载所有需要被保留的 narinfo 文件，才能判断哪些 nar 文件应该被删除。

限制

nix-gc-s3 和 nix copy **不应该同时运行**。

我一般在固定的 systemd 服务中运行这两个服务，服务的脚本中使用 flock 加互斥锁。

正因如此，该方案只适合由一台机器专门跑 Hydra 上传 binary cache 和做 GC。

总结

该方案作为 **土制灵车**，确实**极低成本** + **几乎 Serverless** + **极佳兼容性**。

局限性：基本只适用于配合自建 Hydra CI 一起使用。

1. 没有基于时间的 GC，必须自己维护 GC roots；
2. GC 时要求 binary cache 中的内容必须在本地都有；
3. 只适合由一台机器专门上传 binary cache 和做 GC；
4. 将 Hydra CI 考虑在内，不再 Serverless，但服务器故障不影响 cache 可用性。

其他方案：[github:Mic92/niks3](https://github.com/Mic92/niks3)。

谢谢!



博客文章: blog.linyinfeng.com/posts/homemade-nix-s3-cache